

RetailEdge Inc.

AWS Architecture

Three-Tier

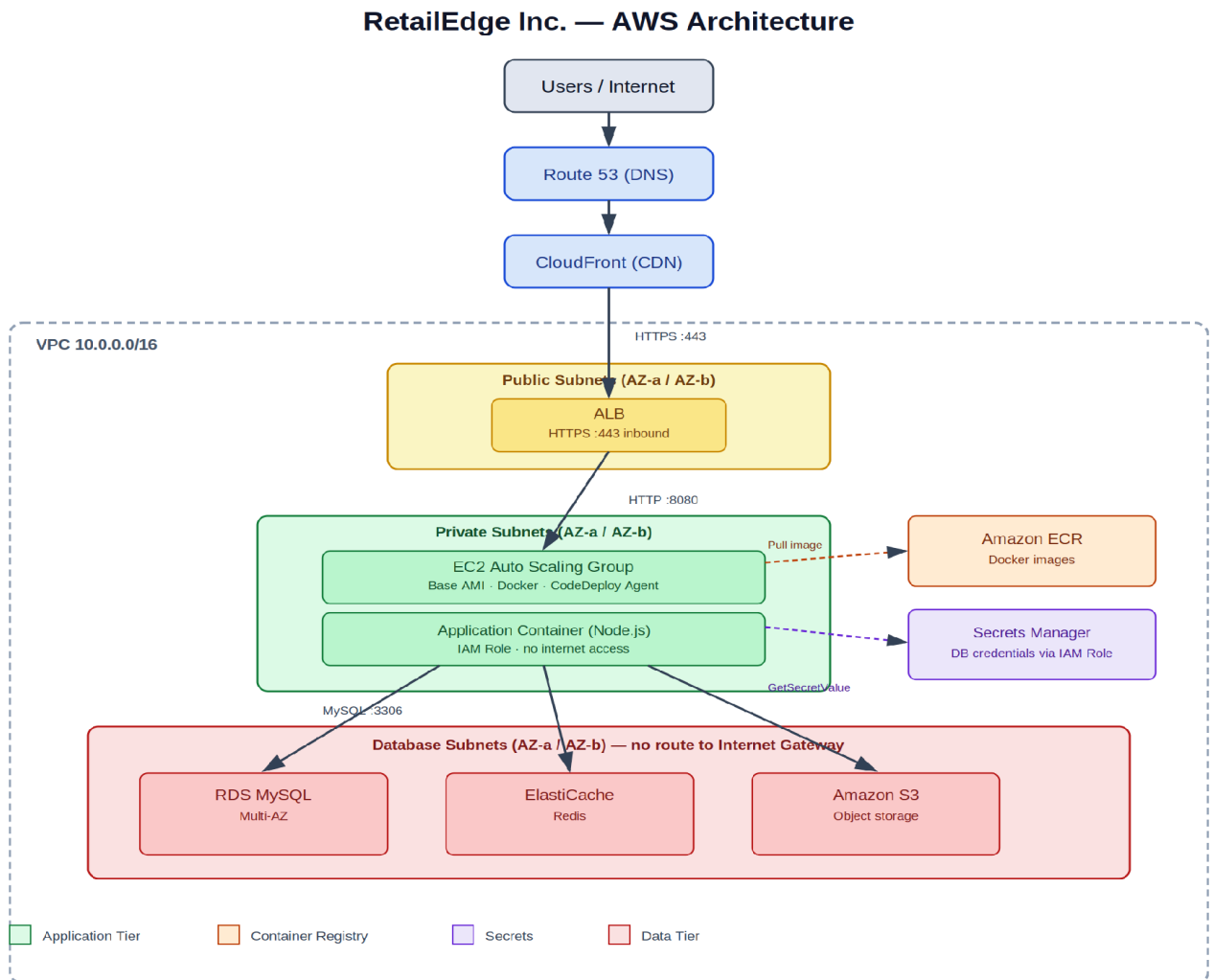
This document describes the AWS architecture for migrating RetailEdge Inc.'s e-commerce platform from three bare-metal servers to a highly available, containerized, three-tier architecture on AWS, including deployment, CI/CD, infrastructure-as-code, and monitoring.

1. Architecture Overview

The solution is organized into three main layers:

- Web Tier — DNS, content delivery, and the public entry point
- Application Tier — Auto-scaled EC2 instances running the application as a Docker container
- Data Tier — Managed database, caching, and object storage services

2. High-Level Diagram



3. Web Tier

- Amazon Route 53 provides DNS resolution for the application domain.
- Amazon CloudFront serves as the content delivery layer.
- An Application Load Balancer (ALB) is the primary public entry point and accepts HTTPS traffic (port 443) from the internet.
- The web tier holds all internet-facing components; the application and database tiers remain isolated from direct internet access.

4. Application Tier

- The ALB distributes traffic across an EC2 Auto Scaling Group hosting the application.
- Application instances sit in private subnets and are not directly reachable from the internet.
- Auto Scaling adjusts capacity automatically to handle demand spikes such as Black Friday.
- The application is packaged as a Docker container; images are stored in Amazon Elastic Container Registry (ECR).
- A base AMI (used by the EC2 Launch Template) contains runtime components such as Docker and the CodeDeploy agent — not application releases.
- AWS CodeDeploy deploys new application versions: instances pull the required image from ECR and run the new container.
- Deployments use a controlled strategy with health validation and automatic rollback on failure.
- Each instance uses a least-privilege IAM role to reach AWS services — no long-lived AWS credentials stored on the instance.

5. Data Tier

- Amazon RDS for MySQL (Multi-AZ) is the primary relational database, improving availability.
- ElastiCache for Redis provides application-level caching.
- Amazon S3 provides object and file storage.
- AWS Secrets Manager securely stores database credentials.
- The application inside the container retrieves credentials at runtime via the EC2 IAM role and the Secrets Manager API — never stored in source code, the Docker image, the AMI, or User Data.
- After retrieval, the application uses the credentials to connect to RDS MySQL.
- Database resources are isolated from direct internet access and accept only required application traffic.

6. Network Architecture

A single VPC (10.0.0.0/16) spans two Availability Zones for segmentation and high availability, with three subnet layers. Controlled outbound connectivity and VPC endpoints (e.g., for Secrets Manager and ECR) can be used where required.

- Public subnets: internet-facing resources such as the ALB
- Application subnets: the EC2 application tier
- Database subnets: the RDS database tier (no route to the Internet Gateway)

Subnet Layout

Layer	Availability Zone A	Availability Zone B
Public	10.0.1.0/24	10.0.2.0/24
Application	10.0.11.0/24	10.0.12.0/24

Database	10.0.21.0/24	10.0.22.0/24
----------	--------------	--------------

7. Security Architecture

Security follows the principle of least privilege: resources may only communicate over the ports and with the components they explicitly require.

Security Group Rules

Security Group	Port	Allowed Source	Purpose
ALB SG	443 (HTTPS)	Internet (0.0.0.0/0)	Public entry point
Application SG	8080 (HTTP)	ALB SG only	App tier traffic
RDS SG	3306 (MySQL)	Application SG only	Database access

- The application tier uses an EC2 IAM role scoped to only what the application and deployment components need (e.g., retrieving the DB secret from Secrets Manager).
- Database subnets have no route to the Internet Gateway, adding a further layer of isolation.
- Security Groups provide stateful, resource-level control; Network ACLs can add subnet-level control where required.

Application Request Flow

Step	From	To	Protocol / Port
1	Internet	ALB	HTTPS : 443
2	ALB	EC2 Application Tier	HTTP : 8080
3	EC2 Application Tier	RDS MySQL	MySQL : 3306

Database Credential Flow

Step	From	To	Action
1	EC2 Application Tier	AWS Secrets Manager	AWS SDK + IAM Role: GetSecretValue
2	AWS Secrets Manager	EC2 Application Tier	Returns DB credentials
3	EC2 Application Tier	RDS MySQL	Authenticated connection : 3306

8. Container and Deployment Architecture

The application artifact is separated from the EC2 server image, so new releases don't require a new AMI.

Base AMI Contents

- Amazon Linux

- Docker
- CodeDeploy Agent
- Required system configuration

Application Packaging

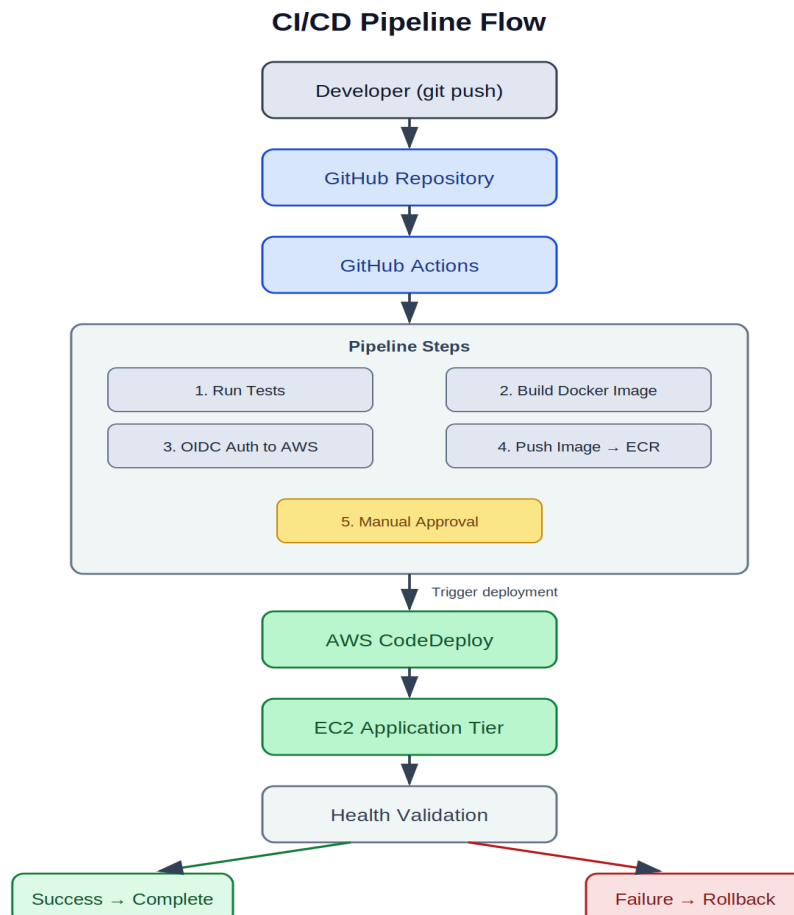
Step	Action	Output	Destination
1	Application build	Source artifact	Docker build context
2	Docker build	Docker image	Amazon ECR
3	CodeDeploy trigger	Pull + run image	EC2 instances

The Auto Scaling Group continues to use the base AMI via the Launch Template; application versions are managed independently through the container image and deployment process.

9. CI/CD Architecture

- Application source code lives in a dedicated application repository (app, Dockerfile, tests, deployment config).
- Infrastructure lives separately in a Terraform repository.
- GitHub Actions authenticates to AWS using OpenID Connect (OIDC) — obtaining temporary credentials via an IAM role instead of storing long-lived AWS access keys in GitHub.

Pipeline Flow



Pipeline Stages

Stage	Action	Notes
1	Test	Automated test suite runs on every push
2	Build	Docker image built from the application repo
3	Authenticate	OIDC-based AWS authentication (no static keys)
4	Push	Image pushed to Amazon ECR
5	Approve	Manual approval gate before deployment
6	Deploy	AWS CodeDeploy rolls out to the EC2 tier
7	Validate	Health checks; automatic rollback on failure

10. High-Level Request Flow (Summary)

- Users → Route 53 → CloudFront → Application Load Balancer
- Application Load Balancer → EC2 Auto Scaling Group → Application Containers
- Application Containers → Data Services:
 - RDS MySQL (Multi-AZ)
 - ElastiCache for Redis
 - Amazon S3

Database credentials are managed separately through AWS Secrets Manager and retrieved by the application at runtime using the EC2 IAM role.

11. Infrastructure as Code

Terraform provisions and manages the AWS infrastructure. Terraform owns the infrastructure lifecycle; the application CI/CD pipeline owns building and deploying application versions.

Terraform-Managed Resources

Category	Resources
Networking	VPC, Subnets, Route Tables, Internet Gateway, NAT / VPC Endpoints as required
Compute	ALB, Target Groups, Launch Template, Auto Scaling Group
Deployment	ECR, IAM Roles and Policies
Data & Secrets	Secrets Manager, RDS MySQL, ElastiCache Redis, S3
Observability	CloudWatch, SNS

12. Monitoring and Operational Architecture

Amazon CloudWatch monitors the health and performance of the application infrastructure. CloudWatch Alarms notify via Amazon SNS when thresholds are exceeded, allowing issues to be detected before they significantly impact customers.

Key Metrics and Alarms

Component	Metric	Notification
ALB	Latency	CloudWatch Alarm → SNS
ALB	5xx errors	CloudWatch Alarm → SNS
RDS	CPU utilization	CloudWatch Alarm → SNS
ElastiCache	Cache hit rate	CloudWatch Alarm → SNS
EC2 / App	Health metrics	CloudWatch Alarm → SNS
Deployment	Application logs	CloudWatch Logs (as required)

13. Design Scope

This document represents the current high-level, network, container, deployment, and security design. The following implementation-level details are defined during the detailed design and implementation phases:

- Route tables, NAT requirements, and VPC endpoints
- Network ACL rules
- Availability Zone placement of individual resources
- Instance sizing and Auto Scaling policies
- Docker configuration and ECR repository configuration
- CodeDeploy deployment groups and hooks
- Secrets Manager integration details
- IAM policies
- Monitoring thresholds and logging
- Backup and rollback configuration
- CI/CD workflow implementation